The 4 Kinds of Gradio Interfaces

So far, we've always assumed that in order to build an Gradio demo, you need both inputs and outputs. But this isn't always the case for machine learning demos: for example, *unconditional image generation models* don't take any input but produce an image as the output.

It turns out that the `gradio.Interface` class can actually handle 4 different kinds of demos:

1. **Standard demos**: which have both separate inputs and outputs (e.g. an image classifier or speech-to-text model)
2. **Output-only demos**: which don't take any input but produce on output (e.g. an unconditional image generation model)
3. **Input-only demos**: which don't produce any output but do take in some sort of input (e.g. a demo that saves images that you upload to a persistent external database)
4. **Unified demos**: which have both input and output components, but the input and output components *are the same*. This means that the output produced overrides the input (e.g. a text autocomplete model)

Depending on the kind of demo, the user interface (UI) looks slightly different:

Let's see how to build each kind of demo using the `Interface` class, along with examples:

Standard demos

To create a demo that has both the input and the output components, you simply need to set the values of the `inputs` and `outputs` parameter in `Interface()`. Here's an example demo of a simple image filter:

```
import numpy as np
import gradio as gr

def sepia(input_img):
    sepia_filter = np.array([
        [0.393, 0.769, 0.189],
        [0.349, 0.686, 0.168],
        [0.272, 0.534, 0.131]
    ])
    sepia_img = input_img.dot(sepia_filter.T)
    sepia_img /= sepia_img.max()
    return sepia_img

demo = gr.Interface(sepia, gr.Image(), "image")
demo.launch()
```

Loading...

gradio/sepia_filter built with Gradio. Hosted on Spaces

Output-only demos

What about demos that only contain outputs? In order to build such a demo, you simply set the value of the `inputs` parameter in `Interface()` to `None`. Here's an example demo of a mock image generation model:

```
import time

import gradio as gr

def fake_gan():
    time.sleep(1)
```

```
    images = [
            "https://images.unsplash.com/photo-1507003211169-0a1dd7228f2d?ixlib
            "https://images.unsplash.com/photo-1554151228-14d9def656e4?ixlib=rb
            "https://images.unsplash.com/photo-1542909168-82c3e7fdca5c?ixlib=rb
    ]
    return images

demo = gr.Interface(
    fn=fake_gan,
    inputs=None,
    outputs=gr.Gallery(label="Generated Images", columns=[2]),
    title="FD-GAN",
    description="This is a fake demo of a GAN. In reality, the images are randc
)

demo.launch()
```

Loading...

gradio/fake_gan_no_input built with Gradio. Hosted on Spaces

Input-only demos

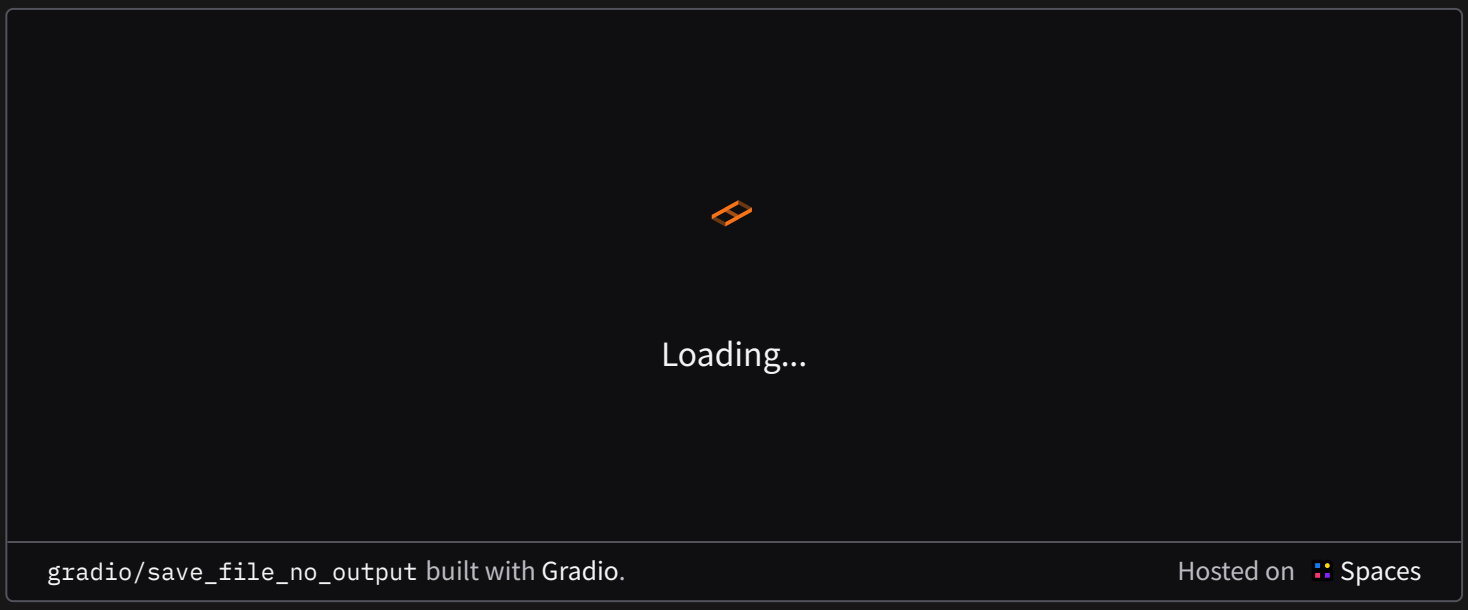
Similarly, to create a demo that only contains inputs, set the value of `outputs` parameter in `Interface()` to be `None`. Here's an example demo that saves any uploaded image to disk:

```
import random
import string
import gradio as gr

def save_image_random_name(image):
    random_string = ''.join(random.choices(string.ascii_letters, k=20)) + '.pnc
```

```
    image.save(random_string)
    print(f"Saved image to {random_string}!")

demo = gr.Interface(
    fn=save_image_random_name,
    inputs=gr.Image(type="pil"),
    outputs=None,
)
demo.launch()
```



Unified demos

A demo that has a single component as both the input and the output. It can simply be created by setting the values of the `inputs` and `outputs` parameter as the same component. Here's an example demo of a text generation model:

```
import gradio as gr
from transformers import pipeline

generator = pipeline('text-generation', model = 'gpt2')

def generate_text(text_prompt):
  response = generator(text_prompt, max_length = 30, num_return_sequences=5)
  return response[0]['generated_text']

textbox = gr.Textbox()

demo = gr.Interface(generate_text, textbox, textbox)
```

```
demo.launch()
```

Loading...

gradio/unified_demo_text_generation built with Gradio. Hosted on Spaces

It may be the case that none of the 4 cases fulfill your exact needs. In this case, you need to use the `gr.Blocks()` approach!

← Reactive Interfaces

Blocks And Event Listeners →

Status